

Introduction

The Parking Management System is a C++ project designed to manage vehicle parking operations efficiently. It allows users to park vehicles, remove vehicles, calculate parking charges, and display parking status. Additionally, the system maintains persistent records using file handling, ensuring that data is not lost after exiting the program.

Objectives

- ☐ Provide a simple and efficient parking management system.
- ☐ Track vehicles with details like registration, owner, and entry time.
- ☐ Calculate charges accurately based on parking duration.

- ☐ Ensure reliable data storage and retrieval using file handling.

System Features

- ☐ Vehicle parking with slot allocation.
- ☐ Vehicle removal with charge calculation
- ☐ Per hour charge system (minimum 1 hour).
- ☐ Display of all parked vehicles with details.
- ☐ File handling to save and load data automatically.
- ☐ Handles both Cars and Bikes.

Code Implementation:

Header file used:

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <ctime>
#include <string>
```

#include <iostream> For input and output operations (cin, cout).

#include For formatted output using setw and alignment.

#include <ctime> For handling entry and exit times.

#include <string>

Class Definition and Structures:

```
class Vehicle {
public:
    string regNo;
    string type;
    string ownerName;
    int slotNo;
    time_t entryTime;

    Vehicle() {
        regNo = "";
        type = "";
        ownerName = "";
        slotNo = -1;
        entryTime = 0;
    }
};
```

Class Vehicle: Stores vehicle details including registration number, type, owner name, slot number, and entry time. The class includes a default constructor to initialize these values.

```
struct Slot {  
    int slotNo;  
    bool isFree;  
};
```

Structure Slot: Stores parking slot number and availability status (free/occupied).

File Handling:

The system uses ofstream and ifstream for file handling. Data about parked vehicles is saved in 'parking.txt'. The saveData() function writes vehicle data to the file, while loadData() reads it back when the program starts, ensuring persistence of data across sessions.

```

// function to save vehicle data into file
void saveData() {
    ofstream file("parking.txt");
    for (int i = 0; i < vehicleCount; i++) {
        file << vehicles[i].regNo << " "
              << vehicles[i].type << " "
              << vehicles[i].ownerName << " "
              << vehicles[i].slotNo << " "
              << vehicles[i].entryTime << endl;
    }
    file.close();
}

// function to load vehicle data from file
void loadData() {
    ifstream file("parking.txt");
    vehicleCount = 0;
    while (file >> vehicles[vehicleCount].regNo
           >> vehicles[vehicleCount].type
           >> vehicles[vehicleCount].ownerName
           >> vehicles[vehicleCount].slotNo
           >> vehicles[vehicleCount].entryTime)
    {
        slots[vehicles[vehicleCount].slotNo - 1].isFree = false;
        vehicleCount++;
    }
    file.close();
}

```

Functions and Their Use:

Function	Purpose
initSlots()	Initializes all parking slots as free.
saveData()	Saves current vehicles' data into file.
loadData()	Loads previous vehicles' data from file.
parkVehicle()	Takes user input and parks a vehicle in a free slot.
removeVehicle()	Removes a vehicle, calculates charges, and frees the slot.
showStatus()	Displays all currently parked vehicles with details.

Function to park a vehicle:

```

void parkVehicle() {
    if (vehicleCount >= MAX_SLOTS) {
        cout << "No free slots available!" << endl;
        return;
    }

    string regNo, type, owner;
    cout << "Enter Registration Number: ";
    cin >> regNo;
    cout << "Enter Vehicle Type (Car/Bike): ";
    cin >> type;
    cout << "Enter Owner Name: ";
    cin >> owner;
}

```

Function to remove a vehicle:

```

// function to remove a vehicle
void removeVehicle() {
    string regNo;
    cout << "Enter Registration Number to remove: ";
    cin >> regNo;

    int index = -1;
    for (int i = 0; i < vehicleCount; i++) {
        if (vehicles[i].regNo == regNo) {
            index = i;
            break;
        }
    }
}

```

Duration and fare calculation:

```

time_t exitTime = time(0);
double hours = difftime(exitTime, vehicles[index].entryTime) / 3600.0;
if (hours < 1) hours = 1; // minimum 1 hour
double charge = hours * CHARGE_PER_HOUR;

cout << "Vehicle Removed:\n";
cout << "Reg: " << vehicles[index].regNo
    << ", Type: " << vehicles[index].type
    << ", Owner: " << vehicles[index].ownerName << endl;
cout << "Total Hours: " << (int)hours << endl;
cout << "Parking Charge: " << charge << " BDT\n";

```

Function to show parking status:

```

void showStatus() {
    cout << left << setw(12) << "RegNo"
        << setw(10) << "Type"
        << setw(15) << "Owner"
        << "Entry Time\n";
    cout << "-----\n";
    for (int i = 0; i < vehicleCount; i++) {
        cout << left << setw(12) << vehicles[i].regNo
            << setw(10) << vehicles[i].type
            << setw(15) << vehicles[i].ownerName
            << ctime(&vehicles[i].entryTime);
    }
    cout << "-----\n";
    cout << "Free Slots: " << (MAX_SLOTS - vehicleCount) << endl;
}

```

Main Function:

```

int main() {
    initSlots();    // set all slots free initially
    loadData();    // load any old data from file

    int choice;
    while (true) {
        cout << "\n===== Parking Management System =====\n";
        cout << "1. Park Vehicle\n";
        cout << "2. Remove Vehicle\n";
        cout << "3. Show Parking Status\n";
        cout << "4. Exit\n";
        cout << "Choose an option: ";
        cin >> choice;

        if (choice == 1) parkVehicle();
        else if (choice == 2) removeVehicle();
        else if (choice == 3) showStatus();
        else if (choice == 4) {
            cout << "Exiting... Data Saved.\n";
            break;
        } else {
            cout << "Invalid option!\n";
        }
    }
    return 0;
}

```

The main() function serves as the entry point of the Parking Management System. It creates an object of the system class, displays the menu in a loop, takes the user's choice, and calls the appropriate functions like adding or removing vehicles, calculating charges, or showing details. Finally, it exits the program safely while ensuring data is saved.

Project Working Explanation & Working Process:

- ☐ User inputs vehicle details (registration number, type, owner name).

- System assigns the first available slot and stores entry time.
- On removal, system calculates the total duration using `difftime()` and charges per hour.
- Minimum 1-hour charge is applied even if parked for less than an hour.
- Vehicle removal frees the slot and updates the file.
- Status function displays all active parked vehicles with formatted table output.
- File handling ensures previous records are exported and imported for continued use.

Conclusion

The Parking Management System provides an efficient way to manage small-scale parking spaces. It integrates key programming concepts such as structures, classes, file handling, and formatted output. The system is simple, user-friendly, and

ensures accurate charge calculation and data persistence. It can also store data for future usage. This makes it a practical project for understanding real-world applications of C++ programming.